Self-Healing and Scalable Web Infrastructure on AWS using Application Load Balancer and Auto Scaling

Cloud Computing Project Report with Implementation Output

Domain: Cloud Infrastructure

Services used: Amazon EC2 · Application Load Balancer · Target Group · Auto Scaling · CloudWatch · S3 · IAM

AWS Region: US East (Ohio) us-east-2 · Implementation date: 5–6 September 2026

Name	**GUNDA TEJESWARA RAO**
Roll No.	**2451-23-749-005**
Class	**CSE-IOT**

1. Project Title

Self-Healing and Scalable Web Infrastructure on AWS using Application Load Balancer and Auto Scaling

2. Abstract

Modern web applications must stay available even when servers fail or traffic changes suddenly. Traditional systems depend on manual monitoring and intervention, which leads to downtime, slow recovery and wasted resources. This project builds a self-healing, auto-scaling web infrastructure on Amazon Web Services (AWS). Ubuntu-based Amazon EC2 instances run an Nginx web server and are launched from a Launch Template. An Application Load Balancer distributes incoming traffic across the instances through a Target Group and continuously performs health checks. An Auto Scaling Group maintains the desired number of instances, replaces failed ones, and adds or removes instances according to CPU utilization. Amazon CloudWatch monitors metrics and triggers scaling alarms, Amazon S3 stores static assets and backups, and an IAM Role gives EC2 secure access to S3 without hard-coded credentials.

The purpose of the project is to demonstrate high availability, fault tolerance and cost-efficient scaling with minimal human effort. The outcome is a web application that remains online during instance failures, scales out under heavy load and scales in when demand falls, reducing cost.

3. Introduction

Cloud computing lets organizations rent computing resources on demand instead of maintaining physical servers. A common challenge is keeping applications available while traffic varies and failures occur. A single server is a single point of failure, and fixed capacity is either too small at peak times or wasteful at quiet times. AWS addresses this with Elastic Load Balancing, Auto Scaling and CloudWatch, which together let infrastructure monitor its own health, recover automatically (self-healing) and adjust capacity to demand (elasticity). This project applies these concepts in a simple, low-cost, student-friendly architecture.

4. Key Points of the Project

- **Web application:** Nginx on Ubuntu EC2 instances, each page showing its server name and Availability Zone
- **Application Load Balancer:** the single public entry point, spreading traffic across Availability Zones
- **Health checks:** detect failed instances automatically

- **Self-healing:** the Auto Scaling Group replaces terminated or unhealthy instances
- **CPU-based scaling:** automatic scale-out and scale-in within minimum / desired / maximum capacity
- **Launch Template + User Data:** fully automated, repeatable instance setup
- **Security:** EC2 accepts HTTP only from the ALB Security Group; an IAM Role is used instead of access keys
- **Monitoring and storage:** CloudWatch dashboards and alarms; private S3 bucket for static assets and backups

5. Advantages and Limitations

Advantages	Limitations
High availability: no single point of failure	Scaling is not instant (boot time and cooldown)
Self-healing with zero manual intervention	ALB and extra instances add cost if left running
Elastic: scales out for load, scales in to save cost	Instances must be stateless; data needs S3/RDS/EFS
Better performance through load distribution	Simple CPU-only policy may not fit all workloads
Consistent, automated, repeatable deployments	HTTP only in the demo (no HTTPS / custom domain)
Secure: Security Group chaining and IAM Roles	Needs careful threshold tuning and AWS knowledge

6. System Workflow and Components

A user request travels Users → Application Load Balancer (listener on port 80) → Target Group → EC2 + Nginx in different Availability Zones. CloudWatch collects CPU and health metrics. If an instance becomes unhealthy or CPU rises above the threshold, an alarm makes the Auto Scaling Group launch or replace instances from the Launch Template; the new instance registers itself with the Target Group. When CPU is low, the extra instance is removed (never below the minimum).

Component	Role in the system
Users / Internet	Clients that send HTTP requests to the application through the ALB DNS name.
Application Load Balancer	Public entry point (Layer 7). Its listener on port 80 forwards requests to healthy targets.
Target Group	Registered pool of EC2 instances; runs health checks and routes only to healthy ones.
EC2 + Nginx (Ubuntu)	Application servers in different Availability Zones serving the website and showing server name / AZ.
Launch Template	Blueprint (AMI, instance type, IAM Role, Security

	Group, User Data) used to create instances.
Auto Scaling Group	Maintains min/desired/max capacity, replaces unhealthy instances and applies the scaling policy.
Amazon CloudWatch	Collects CPU and health metrics, raises alarms that trigger scale-out and scale-in.
Amazon S3	Private bucket for static assets and backups, accessed by EC2 through its IAM Role.
IAM Role	Gives EC2 temporary permissions to S3, so no access keys are stored on servers.
VPC, Subnets, Security Groups	Network layer: public subnets in 2+ AZs; EC2 allows HTTP only from the ALB Security Group.

7. Implementation Output (Screenshots with Explanation)

The screenshots below were captured from the screen recording of the working deployment in the AWS Management Console (region US East – Ohio, us-east-2). The deployment consists of two EC2 web servers, WebServer-1 in us-east-2a and WebServer-2 in us-east-2b, registered in the target group HA-Project-TG behind the load balancer HA-Project-ALB. The account banner has been blurred for privacy.

7.1 Target Group registration and health checks

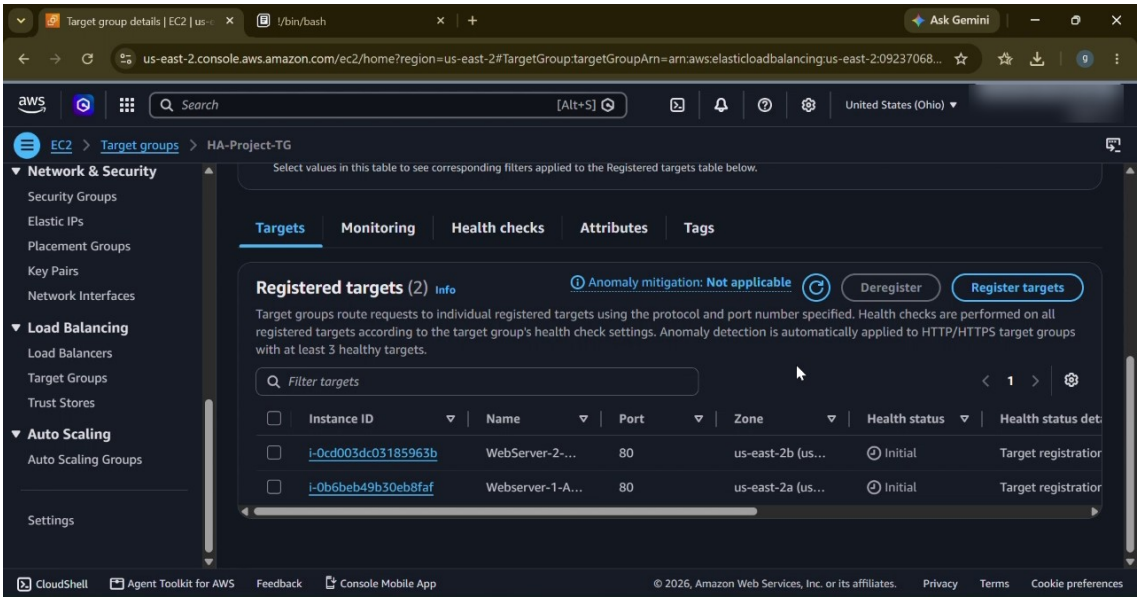


Figure 1: Registered targets of HA-Project-TG right after registration (status: Initial)

Explanation: The target group HA-Project-TG lists two registered targets on port 80: instance i-0cd003dc03185963b (WebServer-2, zone us-east-2b) and i-0b6beb49b30eb8faf (WebServer-1, zone us-east-2a). Their Health status is Initial, which means the load balancer has just registered them and is still running the first health checks. No traffic is sent to a target until it passes these checks.

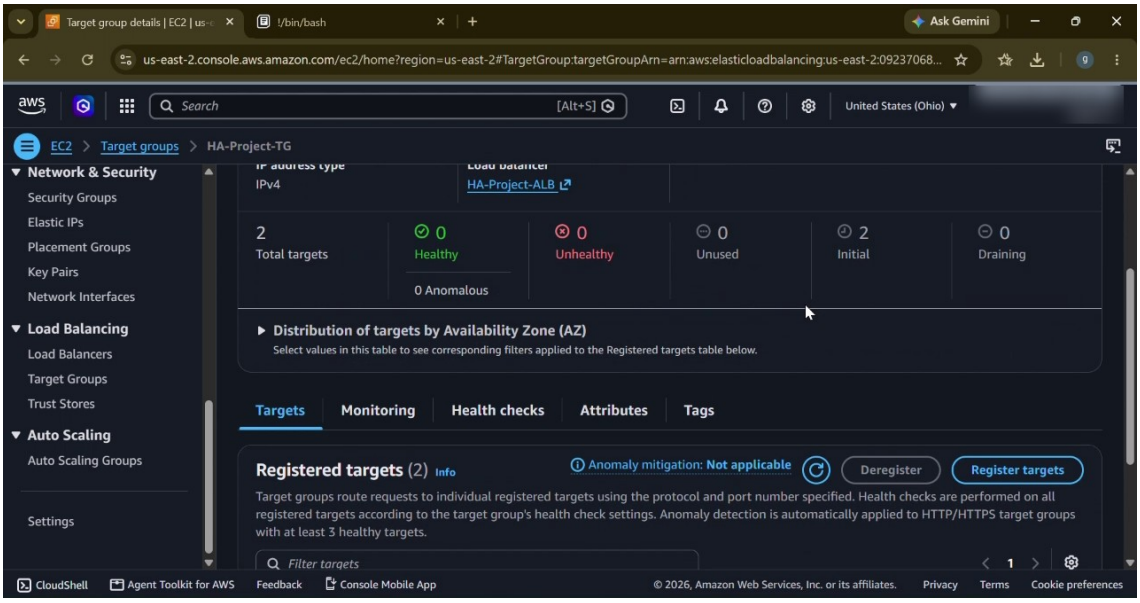


Figure 2: Target group summary: 2 total targets, 0 healthy, 2 initial

Explanation: The summary strip shows Total targets = 2, Healthy = 0, Unhealthy = 0, Unused = 0, Initial = 2, Draining = 0. The target group is linked to the load balancer HA-Project-ALB and uses IPv4. This is the starting state, before the health checks complete.

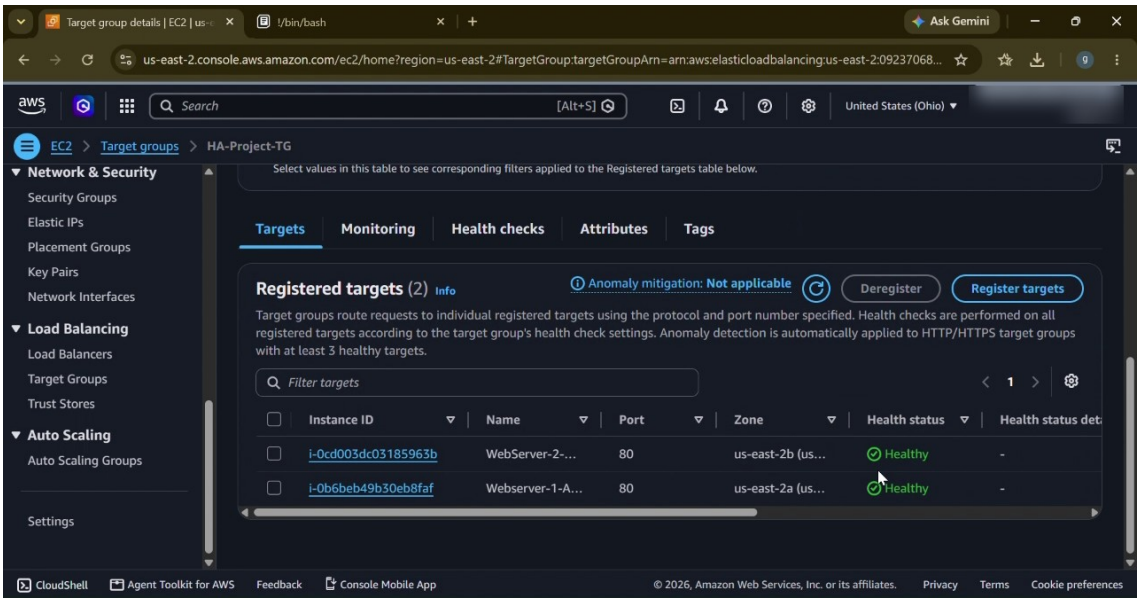


Figure 3: Both targets reported Healthy

Explanation: After a few health-check cycles, both instances change from Initial to Healthy (green tick) with the Health status details field cleared. This proves that Nginx on each server is answering the health-check request on port 80 and that the two servers sit in two different Availability Zones (us-east-2a and us-east-2b), so the application has no single point of failure. The load balancer now sends traffic to both.

7.2 Application Load Balancer creation

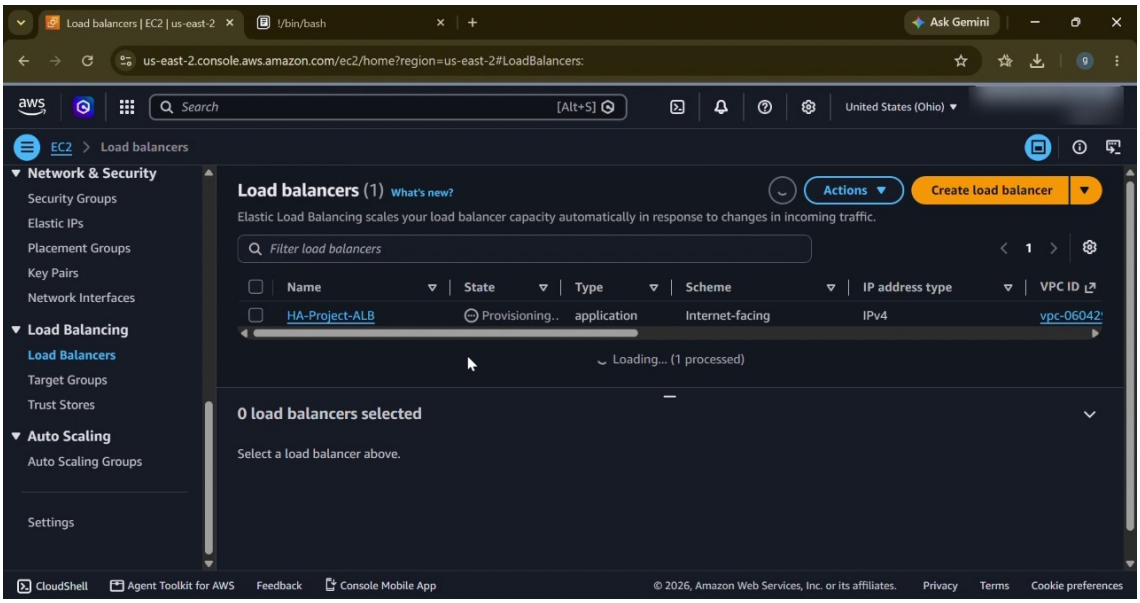


Figure 4: HA-Project-ALB in Provisioning state

Explanation: The Load balancers page lists one load balancer, HA-Project-ALB, of type application and scheme internet-facing with IPv4 addressing. The state is Provisioning, i.e. AWS is still creating the load balancer nodes in the selected subnets.

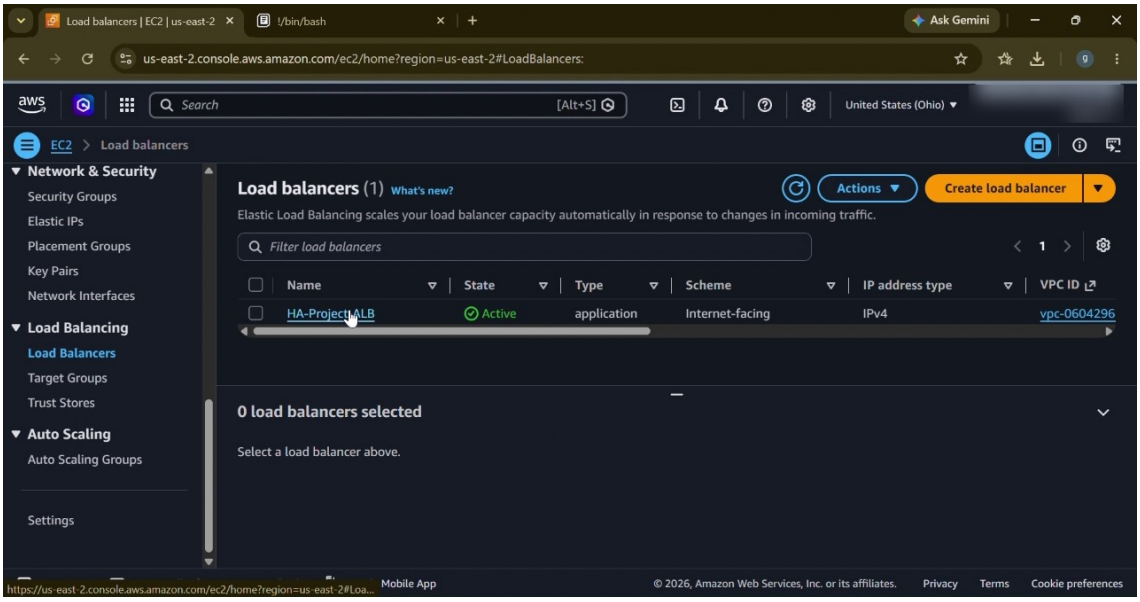


Figure 5: HA-Project-ALB in Active state

Explanation: After provisioning completes, the state changes to Active (green tick). Only an Active load balancer can accept and forward user requests.

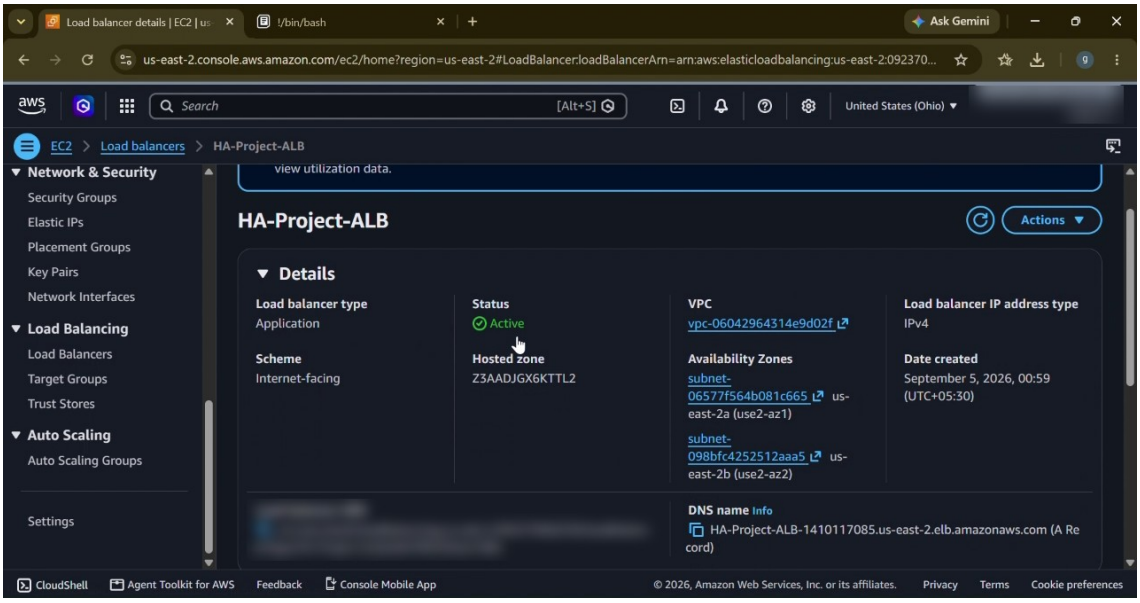


Figure 6: Details of HA-Project-ALB

Explanation: The details page confirms: type Application, scheme Internet-facing, status Active, IP address type IPv4, VPC vpc-06042964314e9d02f, created on 5 September 2026 (00:59 IST). It spans two Availability Zones: us-east-2a (use2-az1) and us-east-2b (use2-az2), each with its own subnet. Spreading the ALB over two zones is what lets the system survive the loss of a whole zone. The DNS name of the load balancer (HA-Project-ALB-1410117085.us-east-2.elb.amazonaws.com) is the public address of the application.

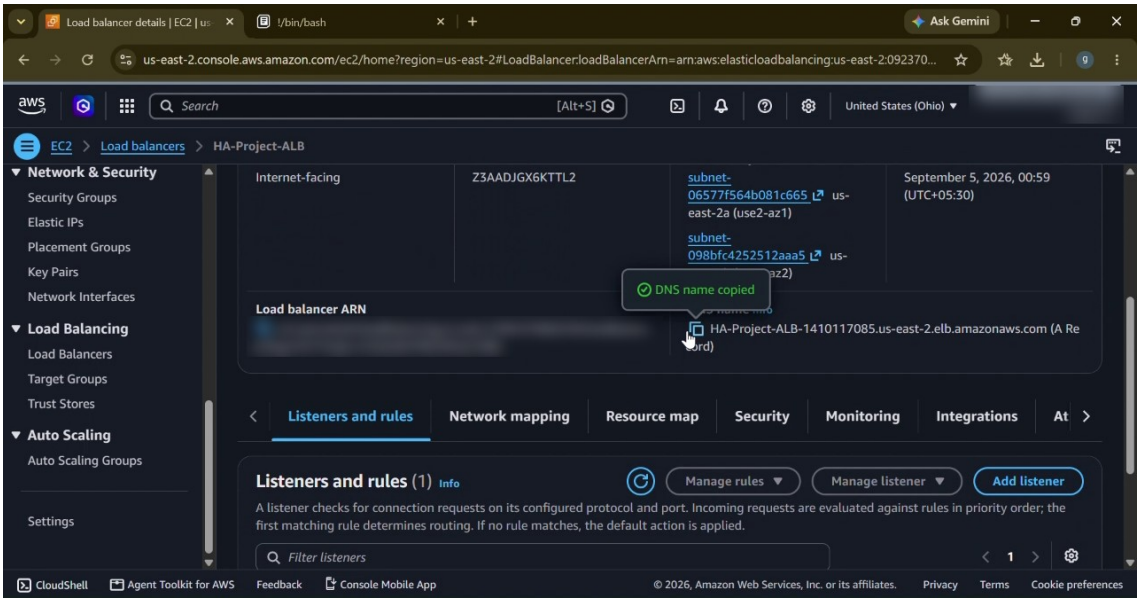


Figure 7: DNS name copied; Listeners and rules (1)

Explanation: The DNS name of the load balancer is copied (message “DNS name copied”) so it can be pasted into a browser. The Listeners and rules tab shows one listener, which accepts HTTP requests and forwards them to the target group HA-Project-TG.

7.3 Web application output through the load balancer

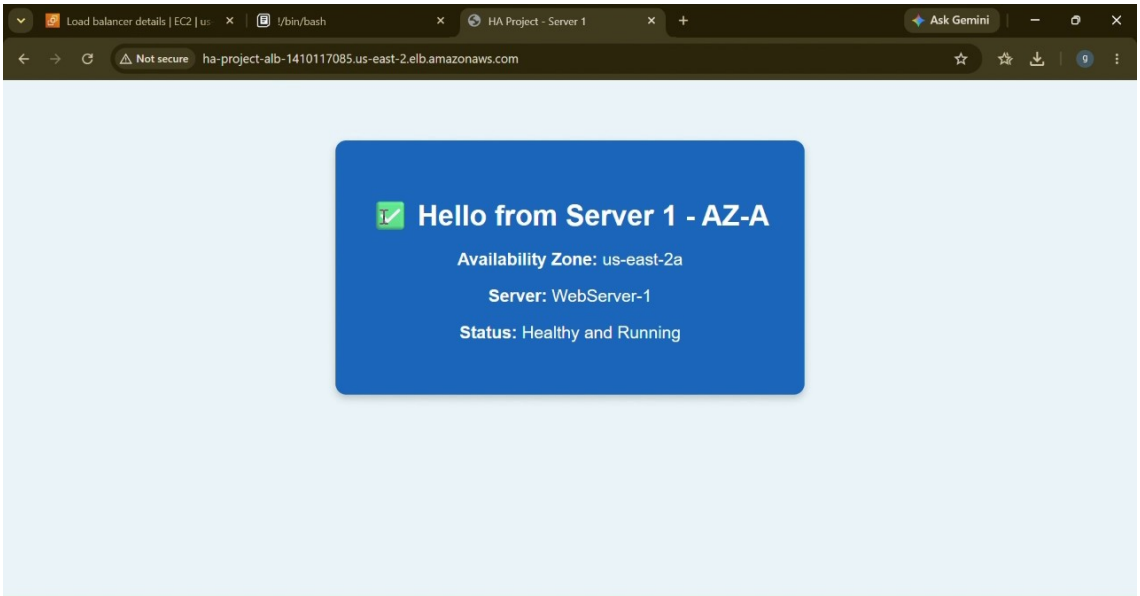


Figure 8: Browser output: Hello from Server 1 – AZ-A

Explanation: Opening the ALB DNS name in a new browser tab (shown as “Not secure” because only HTTP is used) returns the page served by WebServer-1: “Hello from Server 1 – AZ-A”, Availability Zone us-east-2a, Status: Healthy and Running. The address in the browser is the

load balancer, not an individual server, so the user never needs to know which instance answered.

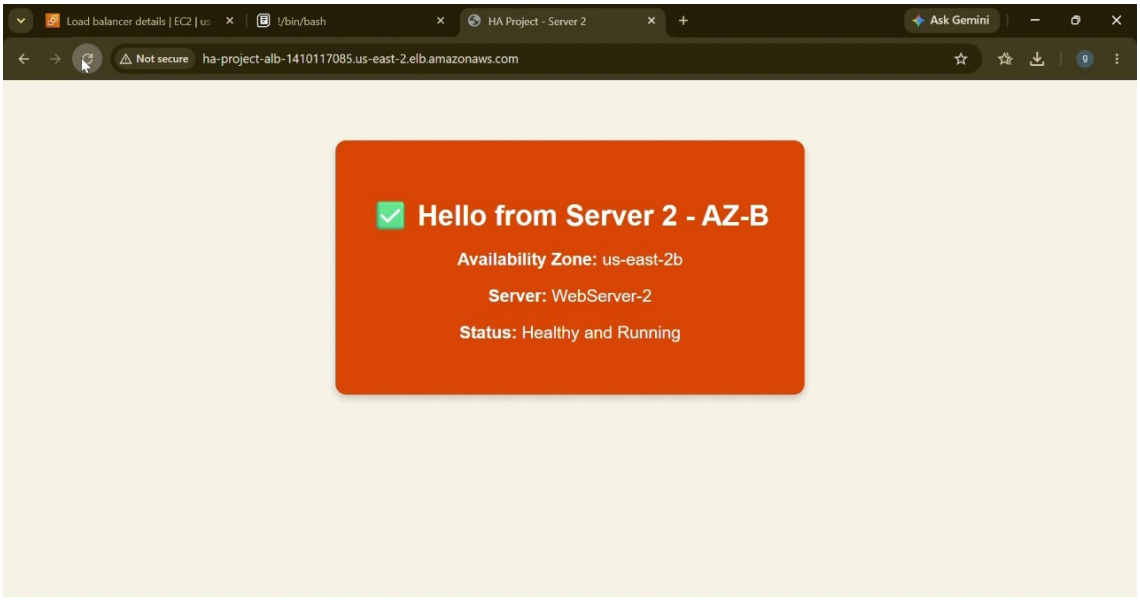


Figure 9: Browser output: Hello from Server 2 – AZ-B

Explanation: On refreshing the same URL, the response comes from the other server: “Hello from Server 2 – AZ-B”, Availability Zone us-east-2b, WebServer-2. The page colour also changes (blue for Server 1, orange for Server 2), which makes the switching easy to see.

8. Result Analysis

In the recording the same ALB URL was refreshed repeatedly and the answer alternated between Server 1 (AZ-A) and Server 2 (AZ-B). This shows:

- **Load balancing:** requests are shared between both servers in a round-robin manner.
- **Multi-AZ high availability:** the two servers run in different Availability Zones, so the site continues if one zone or instance fails.
- **Health checking:** only targets in the Healthy state receive traffic; a failed server is removed automatically, and the Auto Scaling Group (Launch Template + scaling policy) launches a replacement that registers itself in the target group.
- **Single entry point:** users access one stable DNS name irrespective of how many servers exist behind it.

Note: the recording covers the Target Group, Application Load Balancer and web-server output. The Auto Scaling Group, CloudWatch alarms and S3/IAM parts are described in sections 4 and 6 as part of the design.

9. Future Scope

- Enable HTTPS using AWS Certificate Manager and a custom domain with Route 53
- Add a database (Amazon RDS / Aurora Multi-AZ) for dynamic, stateful applications
- Use private subnets with NAT, plus CloudFront and AWS WAF for stronger security
- Adopt target-tracking policies on request count and set up SNS email alerts
- Automate deployment with Terraform / CloudFormation and a CI/CD pipeline

10. Conclusion

The project demonstrates that combining EC2, an Application Load Balancer, Auto Scaling and CloudWatch produces an infrastructure that heals itself and adapts to demand without manual effort. The captured output confirms that two web servers in different Availability Zones were registered in a target group, passed their health checks, and served the application alternately through one load balancer DNS name. Failed instances are replaced automatically, traffic is shared across healthy servers, and capacity grows or shrinks with CPU load, giving high availability, good performance and lower cost.